

3.2 Guía de Funciones: Regresión Logística en R y Python

R — glm + caret

```
library(caret)

data(PimaIndiansDiabetes, package="mlbench")
set.seed(42)

# División
idx <- createDataPartition(PimaIndiansDiabetes$diabetes, p=0.8, list=FALSE)
train <- PimaIndiansDiabetes[idx,]
test  <- PimaIndiansDiabetes[-idx,]

# REGRESIÓN LOGÍSTICA con glm
# -----
modelo_glm <- glm(diabetes ~ .,          # fórmula
                  data = train,
                  family = binomial(link="logit")) # especifica RL
# family=binomial: distribución binomial
# link="logit": función de enlace logit (log-odds)

summary(modelo_glm)
# → Coeficientes, error estándar, z-value, p-value
# Coeficientes significativos (p < 0.05) son relevantes

# Odds Ratios
exp(coef(modelo_glm))      # odds ratio
exp(confint(modelo_glm))   # IC 95% de los OR

# Predicción
probs_glm <- predict(modelo_glm, test, type="response")
# type="response": retorna probabilidades (no log-odds)
```

```

pred_glm <- factor(ifelse(probs_glm >= 0.5, "pos", "neg"),
                  levels=levels(test$diabetes))

confusionMatrix(pred_glm, test$diabetes, positive="pos")

# CURVA ROC y AUC
library(pROC)
roc_obj <- roc(test$diabetes, probs_glm, levels=c("neg","pos"))
auc(roc_obj)      # Área bajo la curva
plot(roc_obj, col="#2E7D32", lwd=2,
     main=paste("ROC - AUC:", round(auc(roc_obj),3)))
abline(0, 1, lty=2, col="gray") # línea aleatoria

# REGULARIZACIÓN con glmnet
library(glmnet)
X_train <- model.matrix(diabetes~., train)[,-1]
y_train <- as.numeric(train$diabetes=="pos")
X_test  <- model.matrix(diabetes~., test)[,-1]

# Ridge (alpha=0), Lasso (alpha=1), ElasticNet (0

```

Python – sklearn + statsmodels

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score, roc_curve
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Escalar (importante para convergencia)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Regresión logística - L2 (Ridge) por defecto
lr = LogisticRegression(
    C=1.0,      # inverso de regularización (C pequeño = más regularización)

```

```
    penalty='l2',      # tipo de regularización
    max_iter=1000,     # máximo de iteraciones de optimización
    random_state=42
)
lr.fit(X_train_s, y_train)

# Coeficientes
feature_names = load_breast_cancer().feature_names
coef_df = pd.DataFrame({'feature': feature_names, 'coef': lr.coef_[0]})
print(coef_df.sort_values('coef', ascending=False).head(10))

# Predicción y métricas
y_prob = lr.predict_proba(X_test_s)[:,-1] # probabilidad de clase 1
y_pred = lr.predict(X_test_s)
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, y_prob):.4f}")

# Curva ROC
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.figure(figsize=(7,5))
plt.plot(fpr, tpr, color='green', lw=2,
         label=f'ROC (AUC={roc_auc_score(y_test,y_prob):.3f})')
plt.plot([0,1],[0,1], '--', color='gray')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('Curva ROC - Regresión Logística')
plt.legend(); plt.tight_layout()
```